

Editorial Pedagógico: Botas Perdidas

Por: alessio

Data: 23 de maio de 2026

Skills: arrays, complexity-analysis, io, loops

▷ Objetivo Pedagógico

- **Laços de Repetição e Fim de Arquivo (EOF):** Compreender como ler e processar dados em um loop contínuo até o encerramento do stream de entrada.
- **Array de Frequência (Mapas de Frequência):** Utilizar arrays para contar ocorrências de dados que possuem um espectro finito e bem definido (tamanhos de 30 a 60).
- **Otimização de Algoritmos:** Evitar abordagens de força bruta $O(N^2)$ ao realizar contagem indireta em tempo $O(N)$.
- **Manipulação de Tipos Mistos:** Ler e associar dados inteiros (tamanho) e caracteres (lados Esquerdo e Direito) simultaneamente.

1. Disciplinas Relacionadas

Disciplina	Tópicos e Aplicações
Algoritmos e Estruturas de Dados I	Arrays unidimensionais, Vetores de Frequência, Contagem e Mapeamento de Ocorrências.
Lógica de Programação	Estruturas condicionais simples, laços de repetição (while , for), e a lógica de processamento até EOF.
Análise de Complexidade	Redução de um problema de tempo quadrático $O(N^2)$ para linear $O(N)$ utilizando espaço auxiliar constante $O(1)$.

array-de-frequencia

laco-de-repeticao

entrada-e-saida

eof

arrays

strings

contagem

logica

otimizacao

2. Editorial e Explicação da Solução

O problema *Botas Perdidas* nos desafia a determinar a quantidade máxima de pares de botas corretos que podem ser formados. Um par correto é definido por possuir uma bota do lado Esquerdo (E) e uma do lado Direito (D), ambas com o exato mesmo tamanho numérico (M).

Para resolver o problema eficientemente, precisamos focar na técnica de **Vetor de Frequência**. Sabendo que os tamanhos das botas variam exclusivamente entre 30 e 60, não é necessário armazenar cada bota individualmente para depois realizar comparações cruzadas (o que custaria $O(N^2)$ comparações).

O Poder do Vetor de Frequência

Um vetor de frequência utiliza o **valor do dado como índice do array**. Ao invés de guardar a bota em si, apenas incrementamos a quantidade de vezes que aquele tamanho apareceu. Como temos botas esquerdas e direitas, basta mantermos dois vetores ou um array matricial, para contar separadamente as ocorrências de cada lado.

A solução correta ocorre nos seguintes passos conceituais:

1. **Leitura e Repetição Contínua:** O problema não nos informa a quantidade exata de casos de teste, mas sabemos que eles se encerram ao final do arquivo (EOF). A estrutura primária é um laço ‘enquanto ler N com sucesso’.
2. **Inicialização dos Contadores:** A cada novo caso de teste (conjunto de N botas), zeramos completamente nossos vetores de frequência (por exemplo, ‘esquerdas[61]’ e ‘direitas[61]’).
3. **Contabilização Mapeada:** Para cada bota processada, lemos seu tamanho M e seu lado L . Se L for ‘E’, incrementamos ‘esquerdas[M]’. Se for ‘D’, incrementamos ‘direitas[M]’.
4. **Processamento dos Pares:** Com todos os dados contabilizados, como formamos pares? Um tamanho só pode formar tantos pares quanto for a quantidade da perna com **menor** número de botas. Em termos matemáticos, o número de pares para um tamanho M é dado por:

$$\text{Pares}(M) = \min(\text{esquerdas}[M], \text{direitas}[M])$$

Basta então criar um iterador do menor tamanho possível (30) ao maior (60) e somar esses mínimos em uma variável totalizadora.

Armadilha Quadrática

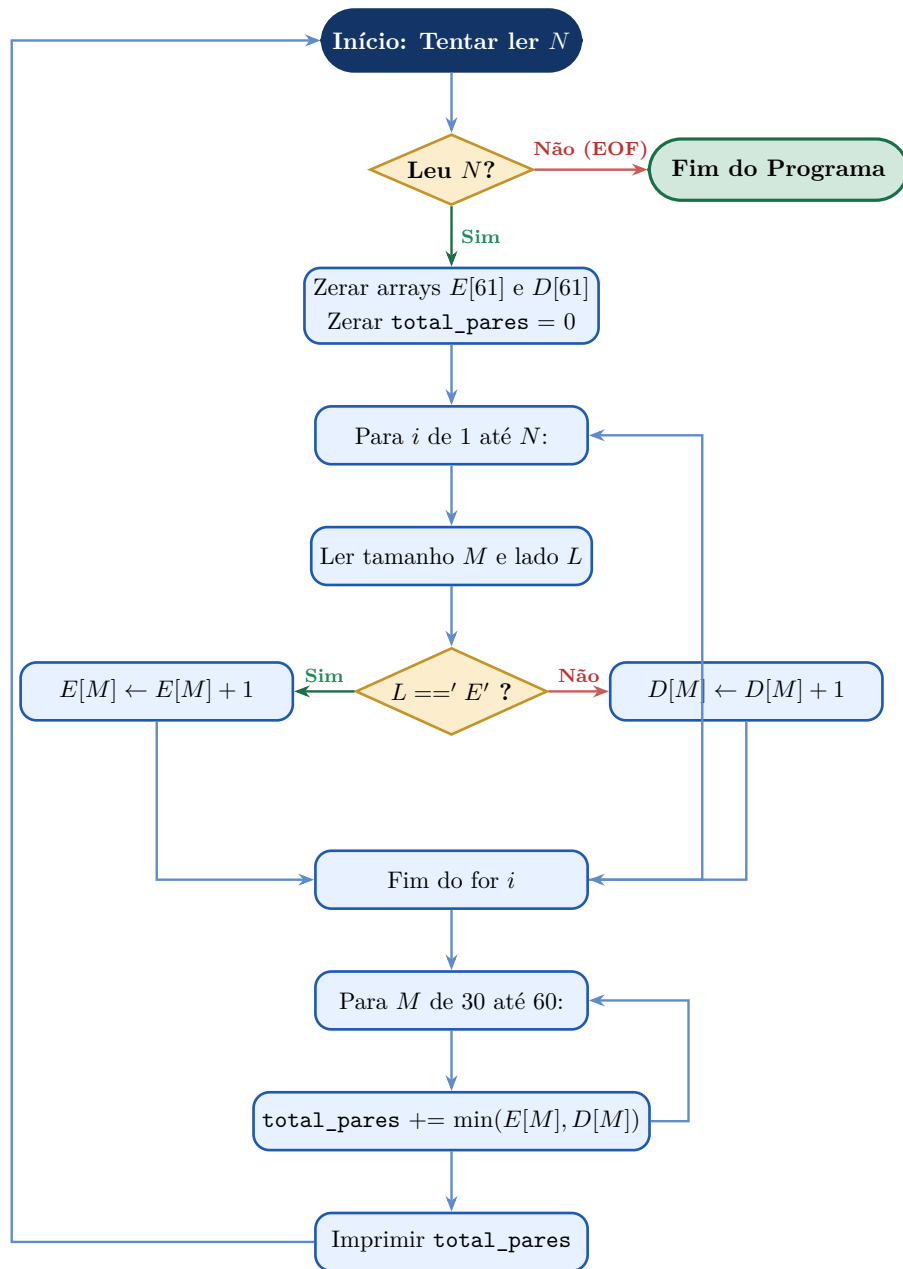
Alunos iniciantes costumam armazenar as botas em dois arrays e depois usar dois laços de repetição **for** aninhados para comparar cada bota esquerda com todas as botas direitas disponíveis (e vice-versa). Esse tipo de solução atinge um número altíssimo de iterações em casos onde N beira o limite estipulado (10.000 botas), gerando o erro de *Time Limit Exceeded*.

Solução Otimizada com Indexação Direta

Os índices das arrays iniciam classicamente em 0. Sabendo que as botas começam no tamanho 30, o desenvolvedor possui duas escolhas de design: (A) Declarar vetores de tamanho 61, utilizando os índices 30 a 60 diretamente, ignorando os índices de 0 a 29 (um custo de memória ínfimo e excelente de ler). (B) Subtrair 30 do tamanho para armazenar nos índices 0 a 30. O método A é muito mais pedagógico.

3. Fluxograma da Solução

Abaixo demonstramos o fluxo do controle para cada iteração do arquivo.



4. Análise de Complexidade

A solução utilizando vetor de frequência elimina buscas de pares. Analisaremos para um único caso de teste composto por N botas.

Complexidade Final: O tempo total para cada caso é $O(N)$, pois a parte fixa $O(1)$ de verificação do laço $30 \dots 60$ não cresce com N . O espaço requerido é estritamente de tamanho constante, resultando em complexidade $O(1)$, independentemente da escala de N .

Etapa	Tempo	Espaço	Justificativa
Inicialização dos arrays	$O(1)$	$O(1)$	Existem tamanhos constantes de arrays (ex.: 61 posições) que precisam ser zerados.
Leitura de N botas	$O(N)$	$O(1)$	Iteramos N vezes. Em cada iteração a complexidade é constante, lendo um número e um caractere, somando 1 à posição do array correspondente.
Contagem final de Pares	$O(1)$	$O(1)$	Existe um laço de tamanho fixo: de 30 a 60 (apenas 31 iterações). Operação de mínimo é $O(1)$.

5. Tutorial Recomendado

Aprofundando os Estudos

Para que o conceito fique cristalino e se torne base fundamental, recomendamos os seguintes recursos:

- **Livro:** *Algoritmos: Teoria e Prática* (Cormen et al.). Capítulo sobre Ordenação em Tempo Linear, que aborda detalhadamente a ideia primária por trás do uso de vetores contadores (Counting Sort).
- **Cplusplus (Arrays):** <https://cplusplus.com/doc/tutorial/arrays/>
- **GeeksforGeeks (Counting Frequencies):** <https://www.geeksforgeeks.org/counting-frequencies-of-array-elements/>

Editorial pedagógico gerado para o Mundo do Código — Pack Técnicas. ID 7. Autor do Problema: alessio.